Slovak University of Technology in Bratislava

Faculty of informatics and information technologies

Reg. No.: FIIT-XXXX-XXXXX TODO

Timon Lumír Fillo

Correlation Power Analysis of Neural Network Weight Parameters

Bachelor's thesis

Thesis supervisor: doc. Hou Xiaolu, Ph.D.

May 2019 TODO

Slovak University of Technology in Bratislava

Faculty of informatics and information technologies

Reg. No.: FIIT-XXXX-XXXXX TODO

Timon Lumír Fillo

Correlation Power Analysis of Neural Network Weight Parameters

Bachelor's thesis

Study programme: Information Security TODO

Study field: Computer Science TODO

Training workplace: Institute of Computer Engineering and Applied Informatics

Thesis supervisor: doc. Hou Xiaolu, Ph.D.

May 2019 TODO

Slovenská technická univerzita v Bratislave
Fakulta informatiky a informačných technológií

ZADANIE BAKALÁRSKEHO PROJEKTU

Meno študenta:
Študijný odbor: Informatika
Študijný program: Informačná bezpečnosť
Názov projektu:

Zadanie:

Práca musí obsahovať:

Anotáciu v slovenskom a anglickom jazyku
Analýzu problému
Opis riešenia
Zhodnotenie
Technickú dokumentáciu
Zoznam použitej literatúry
Elektronické médium obsahujúce vytvorený produkt spolu s dokumentáciou

Miesto vypracovania: Ústav počítačového inžinierstva a aplikovanej informatiky, FIIT STU
Vedúci projektu:

Termín odovzdania práce v zimnom semestri 10. decembra 2019
Termín odovzdania práce v letnom semestri 07. mája 2020

Bratislava 23.09.2019

Declaration of honor

I honestly declare that I prepared this thesis independently, on the basis of consultations and using the literature listed.

In Bratislava, 8.2.2020 .....

Timon Lumír Fillo

Special thanks

Anotácia

Slovenská technická univerzita v Bratislave
Fakulta informatiky a informačných technológií
Študijný program: Informačná bezpečnosť TODO

Autor: Timon Lumír Fillo
Bakalárska práca: Korelačná analýza spotreby váhových parametrov neurónových sietí
Vedúci projektu: doc. Hou Xiaolu, Ph.D.
Máj 2019 TODO

Nasadzovanie neurónových sietí na vstavaných a okrajových zariadeniach zvyšuje význam hodnotenia ich odolnosti voči útokom postranným kanálom. Výpočet inferencie môže prezradiť citlivé interné parametre modelu, najmä váhy, ktoré predstavujú duševné vlastníctvo a ich znalosť uľahčuje ďalšie útoky. Praktické získanie váh z implementácií s pohyblivou rádovou čiarkou je pritom problematické, pretože únik závisí od numerickej reprezentácie a od spôsobu vykonania operácií.

Práca analyzuje princípy korelačnej analýzy príkonu (CPA), existujúce útoky na váhy neurónových sietí a implementáciu sietí na mikrokontroléroch. Navrhuje a implementuje postup na získavanie váh typu float32 z meraní príkonu pri násobení váhy s aktiváciou. Útok je profilovaný: najprv sa naučí únik zo známych náhodných váh a potom hľadá neznáme fixné váhy cieľového zariadenia. Postup je overený experimentálne na platforme ChipWhisperer (CWLITEARM, STM32F3) s jednoduchou doprednou neurónovou sieťou. Hodnotí sa presnosť obnovy váh, efektívnosť (počet potrebných meraní, výpočtová náročnosť) a praktická použiteľnosť vrátane typických chýb, ako je určenie váhy až na mocninu dvoch. Prínosom práce je overený postup extrakcie floating-point váh a analýza jeho obmedzení.

Annotation

Slovak University of Technology in Bratislava
Faculty of informatics and information technologies
Degree Course: Information Security TODO

Author: Timon Lumír Fillo
Bachelor's thesis: Correlation Power Analysis of Neural Network Weight Parameters
Supervisor: doc. Hou Xiaolu, Ph.D.
May 2019 TODO

The growing deployment of neural networks on embedded and edge devices makes it increasingly important to evaluate their resistance to side-channel attacks. Inference computation can leak sensitive internal model parameters, especially the weights. These are intellectual property, and knowing them makes further attacks easier. Recovering weights from floating-point implementations in practice is difficult, because the leakage depends on the numerical representation and on how the operations are executed.

This thesis analyses the principles of correlation power analysis (CPA), existing attacks on neural network weights, and neural network implementations on microcontrollers. It designs and implements an approach for recovering float32 weights from power measurements of the weight–activation multiplication. The attack is profiled: it first learns the leakage from known random weights, then searches for the unknown fixed weights of the target device. The approach is validated experimentally on a ChipWhisperer platform (CWLITEARM, STM32F3) running a small feed-forward neural network. The evaluation covers recovery accuracy, efficiency (number of required traces, computational cost) and practical applicability,

including typical failure modes such as recovering a weight only up to a power of two. The contribution is a validated approach for extracting floating-point weights and an analysis of its limitations.

Abstract

Lay Summary

Table of contents

1 Introduction **1**

1.1 Motivation 1

1.2 Goals of the Thesis 1

1.3 Structure of the Thesis 1

2 Problem Definition **2**

2.1 Side-Channel Leakage of Neural Network Parameters 2

2.2 Threat Model 2

2.3 Research Questions 2

3 Related Work **3**

3.1 Side-Channel Analysis of Neural Networks 3

3.2 Template Attacks 3

4 Solution Overview **5**

4.1 Target Platform 5

4.2 Leakage Model 6

4.3 Profiled Attack 6

4.4 Hierarchical Weight Recovery 7

4.5 Justification of the Chosen Approach 7

5 Risk Assessment **9**

6 Experimental Reproducibility and Integration **10**

6.1 Hardware and Software Requirements 10

6.2 Repository Structure 11

6.3 Reproducing the Experiments 11

6.3.1 Building and Flashing the Firmware 11

6.3.2 Capturing Traces 12

6.3.3 Preprocessing 12

6.3.4 Running the Attack 12

6.4 Data Availability and Open Science 13

7 Sustainability and Environmental Impact **14**

7.1 Long-Term Maintenance 14

7.2 Environmental Impact 15

8 Employability **16**

9 Teamwork, Diversity and Inclusion **17**

10 Scientific Part **18**

11 Evaluation **29**

11.1 Summary of Results 29

11.2 Impact 29

11.3 Future Work 29

Resumé **30**

References **31**

A Technical Documentation

B Work Plan

C Contents of the digital medium

List of Figures

List of abbreviations used

CPA	Correlation Power Analysis
DPA	Differential Power Analysis
HW	Hamming Weight
IEEE	Institute of Electrical and Electronics Engineers
MCU	Microcontroller Unit
NN	Neural Network
PoI	Point of Interest
ReLU	Rectified Linear Unit
SCA	Side-Channel Analysis

Chapter 1

Introduction

1.1 Motivation

1.2 Goals of the Thesis

1.3 Structure of the Thesis

Chapter 2

Problem Definition

2.1 Side-Channel Leakage of Neural Network Parameters

2.2 Threat Model

2.3 Research Questions

Chapter 3

Related Work

3.1 Side-Channel Analysis of Neural Networks

Physical side channels reveal the architecture, inputs and parameters of a neural network [1]. Batina et al. [2] recovered both architecture and weights, but limited weight candidates to a precision of 10^{-2}. Joud et al. [3] and Nozaki and Kobara [4] raised the precision with iterative CEMA and IEEE-754-aware correlation attacks, yet Joud et al. still report small errors rather than exact values. Exact extraction exists under stronger assumptions, using subnormal timing [5] or injected faults [6].

3.2 Template Attacks

A template attack first characterizes the leakage of an identical device and then uses this model against the target [7]. Given a leakage class c, the leakage vector

$\mathbf{t}$ at the points of interest is modeled as

$$\mathbf{t} \mid c \sim \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c), \tag{3.1}$$

and candidates are ranked by likelihood, using several samples and their noise correlation jointly. Later work replaces the Gaussian model with machine-learning models such as two-dimensional CNNs [8]. In our implementation, the covariance matrix is pooled across leakage classes. Wei et al. [9] used a "power template" to recover inputs, but it is a lookup from power features to pixels. To the best of our knowledge, Gaussian templates have not been applied to recovering `float32` weights.

Chapter 4

Solution Overview

4.1 Target Platform

Power traces are measured on a ChipWhisperer-Lite board with an STM32F303 (Arm Cortex-M4) target, controlled over the SimpleSerial protocol. A hardware trigger raised by the firmware just before the forward pass delimits the neural-network inference. Each trace contains 24,400 samples, but only its first 1,000 samples, which contain the targeted multiplication, are used for the analysis. This speeds up profiling.

The target runs a feed-forward neural network with four layers of 7, 5, 4 and 3 neurons (ReLU in the hidden layers, sigmoid at the output), zero biases, and all values stored as IEEE-754 single-precision numbers. The first multiplication of the inference is $v = \text{float32}(w^{\star}x)$, where x is the input chosen by the attacker and $w^{\star}$ is the secret first-layer weight. The firmware is compiled with the default `-mfloat-abi=soft`, so every multiplication runs as a software routine that processes the sign, exponent and fraction bits with integer instructions, even though the MCU has a hardware FPU.

4.2 Leakage Model

A single-precision number [10] consists of a sign bit, 8 exponent bits and 23 fraction bits. For a candidate weight w and known input x we model the leakage of the intermediate $v = \text{float32}(wx)$ by the Hamming weight of its m most significant bits $B_m(v)$:

$$c_m(v) = \text{HW}\left(B_m(v)\right). \tag{4.1}$$

Three resolutions are used: coarse ($m = 17$, 18 classes), fine ($m = 23$, 24 classes) and exact ($m = 32$, 33 classes). Fewer modeled bits tolerate small errors in the candidate weight, while more bits make the model more selective.

4.3 Profiled Attack

Profiling. We capture 50,000 traces with weights drawn uniformly from $[-2, 2]$ (refreshed every 100 traces) and random inputs. Since x and the targeted weight are known, v and its class $c_m(v)$ are known for every trace. Five points of interest (POIs) are selected as the samples with the highest absolute Pearson correlation with $\text{HW}(B_{32}(v))$, at least 20 samples apart. For each class c we estimate the mean $\boldsymbol{\mu}_c$ of the POI vectors $\mathbf{t}$ and a single pooled covariance matrix $\boldsymbol{\Sigma}$, which gives the Gaussian templates

$$\mathbf{t} \mid c \sim \mathcal{N}\left(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}\right). \tag{4.2}$$

One template set is built for each of the three resolutions.

Attack. On the target the weight is fixed and unknown, and only the input x_i is controlled (5,000 attack traces are available). For a candidate $\hat{w}$ the hypothetical class $c_i(\hat{w})$ of $\text{float32}(\hat{w}x_i)$ selects the template, and the evidence is accumulated

over the attack traces:

$$L(\hat{w}) = \sum_{i=1}^{N_{\text{attack}}} \log p\left(\mathbf{t}_i \mid c_i(\hat{w})\right). \tag{4.3}$$

The candidate with the highest $L(\hat{w})$ is the recovered weight.

4.4 Hierarchical Weight Recovery

Evaluating all $\approx 2^{32}$ `float32` values is impractical, so the search narrows the candidates in three stages (Table 4.1), each scored with Eq. (4.3) and a finer leakage model.

Table 4.1: Stages of the hierarchical weight recovery.

Stage	Search range	Step	Candidates	Bits (classes)
Coarse	$[-2, 2]$	10^{-4}	40,001	17 (18)
Fine	$\pm 10^{-4}$ per finalist	10^{-6}	201 per finalist	23 (24)
Exact	± 512 ULPs	1 ULP	$\approx$ 22,550	32 (33)

The coarse stage keeps the six best peaks, at least 0.005 apart, as finalists. The fine stage refines each of them and passes the best one on, and the exact stage selects the final `float32` value. Because $\pm\hat{w}\,2^j$ have the same fraction bits as $\hat{w}$, their products differ from the true one only in the sign and exponent bits, and the coarse and fine models cannot reliably separate them. The exact stage therefore also tries both signs and the scalings 2^j, $j \in \{-10, \ldots, 10\}$, within $[-2, 2]$. The experimental results are given in Chapter 10.

4.5 Justification of the Chosen Approach

Template attack. Previous parameter-recovery attacks on neural networks are mostly non-profiled and based on CPA or CEMA [2, 3, 4]: they correlate one hy-

pothetical leakage value with each sample of the trace. Template attacks were so far applied to neural networks mainly for input recovery [9]. The targeted product can take up to 33 Hamming-weight classes, and its software implementation spreads the leakage over several samples of the trace. A template attack models these samples jointly together with their noise correlation and ranks candidates by their likelihood, which suits testing a large number of 32-bit hypotheses. Profiling is realistic because the attacker can program an identical device with chosen weights (Section 4.3).

Hierarchical search. The coarse model uses only the sign, exponent and upper fraction bits, so a candidate close to the true weight already predicts mostly the right class. The likelihood is therefore smooth enough to find promising regions on a coarse grid, and the later stages spend more candidates and a finer model only where needed. This avoids applying the most detailed model to the whole candidate space.

Pooled covariance and few POIs. Many of the 33 classes are rare in the profiling data, so per-class covariance matrices would be estimated from very few traces. A shared covariance matrix and five POIs keep the number of estimated parameters small and the estimation stable with 50,000 profiling traces.

Chapter 5

Risk Assessment

Strengths. Exact `float32` recovery (100 % success from 263 attack traces), inexpensive hardware, and public code and data.

Weaknesses. Only one weight is attacked, on one unprotected target using software floating-point, and profiling and attack share a board.

Opportunities. Extension to more weights and devices, and evaluation of countermeasures.

Threats. Dependence on one board and on changing ChipWhisperer software, and misuse against deployed models.

Mitigation. Results are claimed only for the evaluated setting, and template transfer to a second board is left as future work. Stored traces and pinned library versions keep the analysis runnable without the board (Chapter 6). The attack targets only our own test network, to help designers protect devices.

Chapter 6

Experimental Reproducibility and Integration

All code, firmware and processed data are in a single public repository.[1] The pipeline has four stages (firmware, capture, preprocessing, attack), each run by one command.

6.1 Hardware and Software Requirements

Hardware. A ChipWhisperer-Lite (CWLITEARM) with its STM32F3 target, connected over USB. Only the capture stage needs the hardware; the analysis runs on any computer.

Software.

- Python 3 with the packages pinned in `requirements.txt` (`chipwhisperer` 6.0.0, `numpy`, `scipy`, `matplotlib`, `tqdm`, `jupyter`).

[1] `https://github.com/LubimPiskoty/Correlation-Power-Analysis-of-Neural-Network-Weight-Parameters`

- The `arm-none-eabi-gcc` toolchain and `make`.
- The ChipWhisperer firmware tree, expected at `$HOME/chipwhisperer/firmware/mcu` (variable `FIRMWAREPATH` in the firmware Makefile).

6.2 Repository Structure

`firmware/` C code for the target: the network (`network.c`), its ground-truth weights (`network_config.h`) and the SimpleSerial commands (`main.c`).

`capture/` `capture.py` and its settings in `configs/cw_config.py`.

`analysis/` `preprocess.py` and the Jupyter notebooks of the attack.

`data/` `raw/` (ChipWhisperer projects) and `processed/` (clipped `.npz` files).

`results/` Figures and tables generated by the notebooks.

`thesis/`, `paper/` LaTeX sources.

All capture parameters (number of traces, input range, batch size, seed) are in `cw_config.py`. The weight range is set by `RANDOM_WEIGHT_MAX` in `main.c` and must match the profiling setting.

6.3 Reproducing the Experiments

6.3.1 Building and Flashing the Firmware

Two firmware builds exist, both with the same code layout. The profiling build generates random weights from a seed; the attack build keeps the fixed unknown weights.

```
cd firmware
```

```
# profiling
make PLATFORM=CWLITEARM CRYPTO_TARGET=NONE
# attack
make PLATFORM=CWLITEARM CRYPTO_TARGET=NONE IS_ATTACK_MODE=1
```

The capture script runs both builds and flashes them itself, so this step is only needed for manual use.

6.3.2 Capturing Traces

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cd capture && python capture.py
```

The script captures 50 000 profiling and 5 000 attack traces with seed 148, using a new random weight set every 100 traces. It writes the ChipWhisperer projects `data/raw/profile` and `data/raw/attack`.

6.3.3 Preprocessing

```
python analysis/preprocess.py
```

The script clips the raw projects to the analysed window (samples 0–1000) and saves `data/processed/profile.npz` and `attack.npz`. It must be rerun after every new capture.

6.3.4 Running the Attack

Run `analysis/TemplateDPA.ipynb` from top to bottom. It selects points of interest, builds the templates, searches for the weight and computes the success rate against the number of traces. Figures are written to `results/figures/`. There is

no test suite; an attack is successful when the recovered weight equals the ground truth in `firmware/network_config.h`.

6.4 Data Availability and Open Science

The source code, firmware and the processed traces (`data/processed/`) are in the public repository, which is enough to rerun the whole analysis without hardware. The raw ChipWhisperer projects are large and excluded from version control. Captures are seeded, but they depend on the physical device, so a new capture reproduces the results statistically rather than trace for trace. Containerisation was not used, because a container cannot easily access the USB-connected ChipWhisperer; the pinned `requirements.txt` fixes the Python environment instead.

Chapter 7

Sustainability and Environmental Impact

7.1 Long-Term Maintenance

The whole work is kept in one version-controlled repository, so every result can be traced to the code and data that produced it. The pipeline has four separate stages (firmware, capture, preprocessing, attack), and each stage has a single entry point. A change in one stage, for example a different network or a new attack notebook, does not require changes in the others as long as the data format stays the same.

Dependencies are pinned in `requirements.txt`, and the capture settings are collected in one configuration file. The clipped traces are stored in the repository as `.npz` files, so the attack can be rerun and extended without the hardware. The main risk to maintenance is the dependence on the ChipWhisperer software and firmware tree, which changes between releases. For this reason the tested version

of the library is fixed, and the board and firmware platform are named explicitly. Further work, such as other platforms or quantized networks, can reuse the analysis part unchanged.

7.2 Environmental Impact

The experiments use a low-power microcontroller board and a laptop, with no large-scale computing. The most expensive step is the trace capture, which runs for a few hours per dataset. The template attack and the success-rate evaluation run on a single CPU. The energy used is therefore small, comparable to normal desktop use.

Storing the processed traces instead of recapturing them avoids repeating the measurements. Only the clipped window is kept, which also reduces storage. No new hardware was built for the work, and the board can be reused for later projects. The wider impact is positive: finding such leakage helps designers protect embedded devices before they are deployed, and so extends their safe lifetime.

Chapter 8

Employability

The work gave me practical experience in embedded security and signal processing. I learned to program and flash a microcontroller in C, to control a ChipWhisperer capture setup, and to run a measurement campaign with a reproducible configuration. On the analysis side, I applied statistical methods (Gaussian templates, point-of-interest selection, success-rate estimation) in Python with NumPy and SciPy.

I also gained general skills: reading and comparing research papers, designing and documenting an experiment, presenting results in figures, and writing a technical text in LaTeX. Working with Git and a pinned environment is a standard practice in industry.

These skills are relevant for roles in hardware and embedded security, security evaluation of devices, and machine-learning engineering on edge devices. Knowledge of how implementation details leak information is also useful in software development in general.

Chapter 9

Teamwork, Diversity and Inclusion

The thesis is an individual work. The author designed the firmware, the capture setup and the analysis, and wrote the text. The supervisor, Xiaolu Hou, set the research direction and gave feedback on the approach and on the results.

Communication with the supervisor was ad hoc, whenever a question or a result needed discussion. The supervisor gave feedback on the work and guided its direction, for example the choice of the threat model and the evaluation, and the decisions were agreed on together. The work plan in Appendix B records how the tasks were divided over time.

The repository is public and documented, so other students can join or continue the work without a personal introduction. The text and the documentation use plain language and avoid unexplained abbreviations, to be readable for people with different backgrounds.

Chapter 10

Scientific Part

Guess My Weight: Profiled Side-Channel Recovery of Floating-Point Neural-Network Weights

Timon Lumír Fillo, Ján Mikulec, Anubhab Baksi,
Jakub Breier, Xiaolu Hou
Slovak University of Technology, Bratislava, Slovakia
Indian Institute of Technology Bhilai, India
TTControl GmbH, Vienna, Austria

***Abstract*—Neural-network parameters deployed on embedded devices may be exposed through physical side-channel leakage during inference. Existing side-channel attacks on floating-point neural-network parameters have often targeted reduced numerical precision, while recovering the complete IEEE-754 representation remains considerably more challenging because of the large and structured 32-bit candidate space. We present a profiled template attack for bit-exact recovery of an IEEE-754 single-precision neural-network weight from power measurements. The attack targets the floating-point multiplication between a known input and a first-layer weight. During profiling, multivariate Gaussian templates are learned from randomized network configurations using Hamming-weight classes of the multiplication result, while the remaining network parameters act as nuisance variables. To efficiently search the structured 32-bit floating-point candidate space, we use a hierarchical coarse-to-fine-to-exact procedure that progressively increases both the numerical and leakage-model resolution. Experiments on a ChipWhisperer-Lite with an Arm Cortex-M4 demonstrate recovery of the exact `float32` representation of the target weight. In the evaluated setting, the attack reaches a bit-exact success rate of 99% with 171 traces and 100% from 263 traces onward. These results demonstrate that profiling can enable practical full-precision extraction of floating-point neural-network parameters from physical leakage.**

***Index Terms*—Template attack, Neural networks, Side-channel attacks**

I. Introduction

Neural-network models deployed on embedded and edge devices often represent valuable intellectual property, as their trained parameters encode knowledge obtained through costly data collection and optimization. At the same time, executing neural-network inference on physical devices can expose model information through side-channel leakage [1], [2]. Power consumption and electromagnetic emanations generated during arithmetic operations may depend on the processed data and model parameters, enabling an adversary with physical measurement capabilities to infer information about the internal model.

Prior work has demonstrated that side-channel leakage can reveal neural-network architectures, processed inputs, and model parameters [3], [4], [5], [6]. Parameter recovery attacks have predominantly relied on non-profiled techniques such as correlation power or electromagnetic analysis, while previous applications of template attacks to neural-network implementations have mainly considered input recovery. Several existing side-channel attacks on neural-network parameters assume a limited numerical precision when recovering floating-point weights [3], [7]. More recent approaches have addressed higher-precision floating-point parameter recovery using iterative or IEEE-754-aware correlation-based techniques [5], [6]. Bit-exact recovery has been shown in [8] where the authors utilized a fault injection attack to reverse-engineer the last layer of the network. In contrast, this work investigates exact 32-bit weight recovery in a profiled setting, where candidate hypotheses are evaluated using learned multivariate leakage templates rather than direct correlation with the measured traces. To the best of our knowledge, a profiled template attack specifically targeting bit-exact 32-bit floating-point neural-network weight recovery has not previously been investigated.

We investigate a profiled template attack that targets the multiplication between a known network input and the first weight of the first layer. During profiling, traces are collected across randomized network configurations for which the corresponding profiling value of the attacked weight is known, while the remaining parameters are treated as nuisance variables. The resulting templates characterize the leakage of the floating-point intermediate $\mathrm{float32}(wx)$ and are subsequently used to evaluate candidate weights on a fixed target network. Weight hypotheses are ranked by their accumulated template likelihood over multiple attack traces, allowing the unknown parameter to be recovered without directly correlating the measured traces with hypothetical leakage values.

A key challenge is that the target parameter is represented as a 32-bit IEEE-754 floating-point value, resulting in a large and finely structured candidate space. To make exact recovery practical, we use a hierarchical coarse-to-fine-to-exact search that progressively increases the resolution of the leakage model. The initial stages use only the most significant parts of the floating-point representation to identify promising regions, while the final stage evaluates neighboring representable `float32` values using the complete 32-bit leakage model. This strategy combines efficient candidate reduction with bit-exact recovery of the target weight.

The main contributions of this work are as follows.

- We present a profiled template attack for recovering an IEEE-754 single-precision neural-network weight from physical side-channel measurements.
- We construct leakage templates across randomized net-

work configurations that can be applied to recover an unknown target weight with no knowledge of the remaining model parameters.
- We introduce a hierarchical coarse-to-fine-to-exact search that enables practical and bit-exact 32-bit weight recovery.
- We experimentally demonstrate bit-exact recovery of the targeted `float32` weight and quantify the attack efficiency through a success-rate analysis, achieving 99% success with 171 attack traces and 100% success from 263 traces onward in the evaluated setting.

II. Background and Related Work

A. Side-Channel Analysis Attacks on Neural Networks

During neural-network [9] inference, each layer transforms its input activations through operations such as weighted multiplications, accumulations, and nonlinear activation functions. In a fully connected layer, for example, the output of a neuron is obtained from a weighted sum of its inputs followed by an activation function. Since these computations repeatedly process network parameters and intermediate activations, their execution may produce data-dependent physical leakage.

Side-channel analysis (SCA) can exploit such leakage to infer sensitive information about a neural-network implementation. Depending on the attack objective, the targeted information may include the network architecture, trained parameters, or data processed by the model. An adversary can observe physical characteristics of the implementation, including execution time, power consumption, or electromagnetic (EM) emissions, while the network performs inference or training. Previous studies have shown that timing leakage can reveal information about activation functions [3], while power and EM measurements can disclose information about network inputs [10] or structural properties of the model, such as the types of executed layers [11].

Particularly relevant to parameter extraction are attacks based on correlation power analysis (CPA) [3], [7], [12]. In CPA, an attacker generates hypothetical intermediate values for different candidate parameters and maps them to predicted leakage values using a suitable leakage model. The predicted leakage is then statistically correlated with the measured power or EM traces. A candidate producing a consistently high correlation is considered more likely to correspond to the secret parameter. The effectiveness of this approach relies on the relationship between the data manipulated during neural-network computation and the resulting physical leakage.

B. Template Attacks

Template attacks are a class of profiled side-channel attacks in which an adversary first characterizes the leakage behavior of a device under controlled conditions and subsequently uses the resulting statistical model to recover an unknown sensitive value [13]. In the classical setting, profiling traces with known intermediate values are used to construct multivariate leakage distributions at selected points of interest. During the attack phase, candidate hypotheses are evaluated according to their likelihood under these templates.

For each leakage class c, the leakage vector $\mathbf{t}$ is commonly modeled using a multivariate Gaussian distribution,

$$\mathbf{t} \mid c \sim \mathcal{N}\left(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c\right),$$

where $\boldsymbol{\mu}_c$ and $\boldsymbol{\Sigma}_c$ denote the class-dependent mean vector and covariance matrix, respectively.

Template attacks have also been considered in the context of neural-network implementations. Wei et al. [4] applied a power template attack to an FPGA-based convolutional neural-network accelerator with the objective of reconstructing the network input from side-channel measurements. Their attack targets information related to the processed image rather than the model parameters.

To the best of our knowledge, previous applications of classical template attacks to neural-network implementations have primarily focused on input recovery, and we are not aware of prior work applying a profiled template attack specifically to the recovery of floating-point neural-network weights. This motivates the study of template-based parameter recovery considered in this work.

C. IEEE-754 Floating-Point Representation

The target implementation represents neural-network weights, inputs, and intermediate values using IEEE-754 single-precision floating-point numbers [14]. A 32-bit floating-point value consists of one sign bit, eight exponent bits, and twenty-three fraction bits,

$$b_{31} \ldots b_0 = \underbrace{b_{31}}_{\text{sign}} \underbrace{b_{30} \ldots b_{23}}_{\text{exponent}} \underbrace{b_{22} \ldots b_0}_{\text{fraction}}.$$

The IEEE-754 standard refers to the lower 23 stored bits as the fraction field. These bits are also commonly referred to informally as the mantissa bits; in this work, we use the term *fraction bits* throughout.

For a normalized value, the represented number is

$$x = (-1)^s \left(1 + \frac{M}{2^{23}}\right) 2^{E-127},$$

where s is the sign bit, E denotes the unsigned exponent field, and M is the integer represented by the fraction bits.

This representation is relevant to side-channel analysis because arithmetic operations manipulate data whose binary encoding depends on the sign, exponent, and fraction fields. In particular, numerically close floating-point values may differ only in a small number of fraction bits, while larger numerical changes may also modify the exponent field. These properties influence the leakage models used for recovering floating-point neural-network parameters.

The hierarchical recovery strategy in this work exploits this structure by increasing the number of considered fraction bits across three stages. A reduced-bit representation is used for the initial search to provide robustness over a wide candidate range, while progressively more bits are included during refinement to distinguish increasingly close floating-point candidates.

III. Threat Model and Experimental Setup

A. Threat Model

We consider a profiled adversary. The adversary knows the network architecture and the position of the targeted multiplication in the forward pass, can choose the network input x, and can measure the power consumption of the target device during inference. For profiling, the adversary has a device of the same type that can be programmed with weights of the adversary's choice, so the targeted weight is known for every profiling trace. On the attacked device, the weights are fixed and unknown. In our experiments, the profiling and attack traces are acquired on the same physical board, which is re-programmed between the two phases.

B. Target Neural Network

We consider a feed-forward neural network implemented in software on the target microcontroller. The network consists of four layers with 7, 5, 4, and 3 neurons, respectively. The first layer represents the input layer, while the following three layers are fully connected. The hidden layers use the ReLU activation function, whereas the output layer applies a sigmoid activation. All weights and intermediate values are represented as IEEE-754 single-precision floating-point numbers.

The objective of the attack is to recover the first weight of the first fully connected layer,

$$w^\star = W^{(1)}_{0,0}.$$

In the fixed target network used during the attack phase, this weight is set to 1.43 in the firmware source. At run time, the device stores the nearest `float32` value, $1.42999994754791\ldots$ (bit pattern `0x3FB70A3D`).

The targeted weight is particularly suitable for a first side-channel analysis because it is involved in a multiplication with an attacker-chosen input. For each inference, the attacker supplies a floating-point value x, which is assigned to the first neuron of the input layer. The remaining input neurons keep their fixed initial activation of 0.5. During the forward pass, the first neuron of the first hidden layer therefore computes a term of the form

$$z^{(1)}_0 = b^{(1)}_0 + w^\star x + \sum_{j=1}^{6} W^{(1)}_{0,j} a^{(0)}_j,$$

where $a^{(0)}_j$ denotes the activation of input neuron j and $b^{(1)}_0$ is the neuron bias. In the implemented network, the biases are initialized to zero. The forward pass processes the neurons in order, starting with the first input of the first neuron. The first multiplication of the whole inference is consequently

$$v = \mathrm{float32}(w^\star x).$$

The target microcontroller (STM32F303) has a hardware floating-point unit, but the firmware is compiled with the ChipWhisperer default `-mfloat-abi=soft`. Every floating-point multiplication is therefore executed by a software library routine that processes the sign, exponent, and fraction of the operands with integer instructions.

This intermediate value is the leakage target considered throughout this work. Since x is known to the attacker, each candidate value $\hat{w}$ induces a hypothetical intermediate

$$\hat{v} = \mathrm{float32}(\hat{w}x),$$

from which a corresponding leakage class can be predicted. The attack therefore does not attempt to observe the weight value directly. Instead, it exploits the data-dependent leakage generated while the processor evaluates the floating-point multiplication between the unknown weight and the known input.

Targeting the first weight also limits uncertainty in the processed operand. For weights in later layers, the corresponding input activation depends on previous weighted sums and nonlinear activation functions and is therefore generally unknown unless earlier network parameters are already available. In contrast, for $W^{(1)}_{0,0}$ one operand of the multiplication is directly selected by the attacker. This makes the first weight a natural target for evaluating the feasibility of profiled template-based neural-network parameter recovery.

Although the analysis focuses on a single weight, the complete forward pass is executed during every measurement. Consequently, the measured power traces contain leakage not only from the targeted multiplication but also from the remaining multiplications, additions, activation functions, and control operations of the network. During profiling, the non-target weights are varied together with the target weight and are not explicitly incorporated into the leakage model. They therefore act as nuisance parameters whose contribution must be tolerated by the profiling procedure.

C. Profiling Scenario

The profiling phase is used to characterize the side-channel leakage of the targeted floating-point multiplication. For this purpose, all network weights are drawn at random from the interval $[-2, 2]$, and a new random configuration is generated every 100 traces (Section III-E).

The targeted weight $W^{(1)}_{0,0}$ is known during profiling. For every acquired trace, the corresponding input value x and the current value of the targeted weight are stored together with the power measurement. This information allows the intermediate product

$$v = \mathrm{float32}(W^{(1)}_{0,0}x)$$

to be reconstructed exactly for the profiling measurements.

The remaining network weights are also randomized, while the biases stay zero in both phases. However, the non-target weights are not used by the leakage model and are not required during template construction. They therefore act as nuisance parameters whose influence is implicitly captured in the measured power traces. This profiling setting is intended to reduce dependence on one particular fixed network configuration.

The profiling dataset contains $50{,}000$ traces. The network configuration is refreshed once every 100 traces. Consequently, the dataset contains 500 distinct randomized network configurations. Within each configuration, the weight values remain fixed while different input values are applied.

Let the profiling dataset be denoted by

$$\mathcal{D}_{\text{prof}} = \{(x_i, w_i, \mathbf{T}_i)\}_{i=1}^{N_{\text{prof}}},$$

where x_i is the known network input, w_i is the known value of the targeted weight, and $\mathbf{T}_i$ is the corresponding measured power trace. In our experiments,

$$N_{\text{prof}} = 50{,}000.$$

For each profiling measurement, the known pair (x_i, w_i) is used to compute the hypothetical intermediate processed by the target device. The corresponding leakage class is then derived from the IEEE-754 representation of the product. These labeled profiling traces are subsequently used for point-of-interest selection and for construction of the multivariate Gaussian templates described in Section IV.

The profiling phase is performed independently from the attack phase. In particular, the fixed target-network weight used during the attack is not required for template construction. The templates characterize the leakage associated with classes of floating-point multiplication results rather than with individual weight values. As a result, the same profiling model can be evaluated against candidate weights of an independently configured target network.

D. Attack Scenario

During the attack phase, the neural-network configuration is fixed and the value of the targeted weight $W_{0,0}^{(1)}$ is unknown to the attacker. The attacker can provide the input value x and acquire the corresponding power trace generated during network inference. Unlike the profiling phase, the value of the targeted weight is not included in the attack-trace metadata.

The attack dataset is represented as

$$\mathcal{D}_{\text{attack}} = \{(x_i, \mathbf{T}_i)\}_{i=1}^{N_{\text{attack}}},$$

where x_i denotes the known input value and $\mathbf{T}_i$ is the corresponding measured power trace. The unknown parameter $w^\star$ remains constant across all traces because the same target network is used throughout the attack experiment.

For a candidate weight $\hat{w}$, the attacker computes the hypothetical intermediate

$$\hat{v}_i = \text{float32}(\hat{w} x_i).$$

The IEEE-754 representation of $\hat{v}_i$ is then mapped to a leakage class according to the leakage model established during profiling. The measured trace $\mathbf{T}_i$ is compared with the template corresponding to this hypothetical class.

Each attack trace therefore provides statistical evidence about the correctness of the candidate weight. Evidence from multiple traces is accumulated using the template log-likelihood,

$$L(\hat{w}) = \sum_{i=1}^{N_{\text{attack}}} \log p\left(\mathbf{t}_i \mid c_i(\hat{w})\right),$$

where $\mathbf{t}_i$ denotes the samples of trace $\mathbf{T}_i$ at the selected points of interest and $c_i(\hat{w})$ is the hypothetical leakage class predicted for candidate $\hat{w}$.

The candidate with the highest accumulated likelihood is considered the most plausible value of the targeted weight. To avoid evaluating the complete IEEE-754 weight space directly, the attack applies the hierarchical coarse-to-fine-to-exact search described in Section IV-E. The search progressively reduces the candidate space while increasing the precision of the leakage model.

For experimental validation, the true weight value is known to the evaluator from the fixed network configuration. This value is used only to assess whether the attack successfully recovered the targeted parameter and is not used during candidate scoring or template matching.

E. Measurement Platform

Power measurements are acquired with a ChipWhisperer-Lite and its integrated STM32F303 (Arm Cortex-M4) target (`CWLITEARM`). The firmware is programmed through the STM32 serial bootloader. The scope uses the ChipWhisperer default settings: the target is clocked at 7.37 MHz, the ADC samples at four times the target clock (29.48 MS/s), and the gain is 25 dB. Communication between the host and the target is implemented using the ChipWhisperer SimpleSerial protocol.

The capture script uses a single fixed acquisition window for every inference. Each trace contains 24,400 samples (about 6,100 target clock cycles), close to the capture-buffer limit of the ChipWhisperer-Lite. No sample decimation is applied, i.e., the decimation factor is set to one. Before collecting either the profiling or attack dataset, 50 dummy inferences are executed to allow the target and measurement setup to reach a stable operating state.

A hardware trigger is used to delimit the neural-network inference operation. Immediately before execution of the forward pass, the firmware raises the trigger signal using `trigger_high()`. The trigger is lowered using `trigger_low()` after the complete forward pass has finished. The acquisition window starts at the rising edge of the trigger and has a fixed length. The acquisition window therefore begins shortly before the targeted multiplication and also contains the subsequent forward-pass operations. The analysis uses only the first 1,000 samples (250 clock cycles) of each trace, which contain the targeted multiplication.

The host supplies the network input through a SimpleSerial `p` command. The input value is represented as an IEEE-754 single-precision floating-point number and packed into the first four bytes of a 16-byte command payload. Upon receiving the command, the firmware assigns this value to the first input neuron and executes the network forward pass.

A separate SimpleSerial `k` command is used to configure the weights. During profiling, the host sends a 32-bit random seed to the target. The firmware initializes its pseudo-random number generator with this seed and generates all network weights uniformly in the interval $[-2, 2]$. The generated configuration remains unchanged for a batch of 100 consecutive measurements. The firmware additionally returns the first weight of the first layer to the host, allowing the attacked

parameter to be associated with the corresponding profiling traces.

During the attack phase, the firmware is rebuilt with the fixed target-network parameters. The same `k` command sequence is retained in order to preserve a similar communication procedure between profiling and attack acquisition. However, the random values generated in response to this command do not replace the fixed attack weights. Thus, only the network input varies between attack measurements, while the unknown target weight remains constant.

The input values used in both phases are generated independently from a uniform distribution,

$$x \sim \mathcal{U}(-1, 1).$$

A fixed host-side random seed is used to make the acquisition procedure reproducible.

The profiling and attack measurements are stored in separate ChipWhisperer projects. The profiling project contains $50{,}000$ traces, while the attack project contains $5{,}000$ traces. For profiling traces, the stored metadata contain both the applied input and the corresponding first-layer target weight. For attack traces, only the applied input is stored, reflecting the assumption that the target weight is unknown during the recovery phase.

IV. Profiled Template Attack

This section presents the proposed profiled template attack for recovering the targeted neural-network weight. Section IV-A defines the floating-point leakage model, Section IV-B describes the selection of points of interest, and Section IV-C explains the construction of the Gaussian templates. Section IV-D introduces the likelihood-based candidate scoring, while Section IV-E describes the coarse-to-fine-to-exact recovery procedure.

A. Leakage Model

The proposed attack targets the intermediate value produced by the floating-point multiplication between the known input x and a candidate weight w. For each trace, the corresponding intermediate is modeled as

$$v = \mathrm{float32}(wx).$$

We assume that the instantaneous power consumption depends on the binary representation of this intermediate value. The leakage is therefore modeled using the Hamming weight of selected bits of the IEEE-754 single-precision representation of v. Let $B_m(v)$ denote the m most significant bits of the IEEE-754 representation of v, i.e., the sign bit, the 8 exponent bits, and the first $m - 9$ fraction bits. The leakage class at resolution m is defined as

$$c_m(v) = \mathrm{HW}\left(B_m(v)\right).$$

To support efficient candidate refinement, three leakage resolutions are used. The coarse model considers the sign bit, the 8 exponent bits, and the 8 most significant fraction bits, resulting in 17 modeled bits and 18 possible Hamming-weight classes. The fine model extends the fraction contribution to 14 bits, resulting in 23 modeled bits and 24 possible classes. The exact model uses the complete 32-bit IEEE-754 representation and therefore contains 33 possible Hamming-weight classes.

The three leakage models can be written as

$$\begin{aligned} c_{\mathrm{coarse}}(v) &= \mathrm{HW}\left(B_{17}(v)\right), \\ c_{\mathrm{fine}}(v) &= \mathrm{HW}\left(B_{23}(v)\right), \\ c_{\mathrm{exact}}(v) &= \mathrm{HW}\left(B_{32}(v)\right). \end{aligned}$$

Using fewer fraction bits in the first stages makes the model tolerant to small errors in the candidate weight. A candidate that is close to, but not exactly equal to, the true weight produces a product whose upper bits mostly agree with the true product, while its lower fraction bits are effectively random. The unmodeled bits still influence the measured leakage, and their contribution becomes part of the noise captured by the template covariance. Increasing the number of modeled bits makes the model more selective, but it also requires the candidate to be closer to the true weight. The full 32-bit model is finally used in the exact stage, where the candidates are individual `float32` values.

During profiling, the true intermediate value is available because both x and the profiling weight are known. During the attack, the same leakage classes are computed hypothetically for each candidate weight $\hat{w}$ as

$$c_m(\hat{w}, x) = \mathrm{HW}\left(B_m\left(\mathrm{float32}(\hat{w}x)\right)\right).$$

These hypothetical classes determine which profiling template is used to evaluate each candidate against the measured attack traces.

B. Point-of-Interest Selection

The points of interest (POIs) are selected from the profiling traces by identifying time samples whose measured leakage is most strongly correlated with the targeted floating-point multiplication. For each profiling trace i, the exact intermediate value

$$v_i = \mathrm{float32}(w_i x_i)$$

is known from the profiling input x_i and profiling weight w_i.

A scalar leakage label is assigned to each profiling trace using the Hamming weight of the complete 32-bit IEEE-754 representation of v_i,

$$h_i = \mathrm{HW}\left(B_{32}(v_i)\right).$$

For every time sample s in the analysis window (the first $1{,}000$ samples of each trace), the Pearson correlation coefficient between the measured trace samples and the leakage labels is then computed as

$$\rho_s = \mathrm{corr}\left(\left(T_1[s], \ldots, T_N[s]\right), \left(h_1, \ldots, h_N\right)\right).$$

The samples are ranked according to the absolute correlation magnitude $|\rho_s|$. To avoid selecting several samples from the same local leakage peak, a minimum temporal separation is

enforced between selected POIs. In the current implementation, five POIs are retained, with a minimum spacing of 20 samples between them.

Let the selected positions be denoted by

$$\mathcal{P} = \{p_1, p_2, \ldots, p_5\}.$$

Each power trace is subsequently reduced to the five-dimensional leakage vector

$$\mathbf{t}_i = [T_i[p_1], T_i[p_2], T_i[p_3], T_i[p_4], T_i[p_5]]\,.$$

The selected POIs are shared by the coarse, fine, and exact template sets. Only the leakage-class definition changes between the three stages, while the same five time samples are used for template construction and candidate evaluation.

C. Template Construction

The profiling traces are grouped according to the leakage classes defined in Section IV-A. Each profiling leakage vector $\mathbf{t}_i$ (Section IV-B) is labeled with the Hamming-weight class $c_i = c_m(v_i)$ of its known intermediate value.

For each class c, the class mean is estimated as

$$\boldsymbol{\mu}_c = \frac{1}{N_c} \sum_{i:c_i=c} \mathbf{t}_i,$$

where N_c denotes the number of profiling traces assigned to class c.

Instead of estimating a separate covariance matrix for every class, the implementation uses a pooled covariance matrix shared across all classes. This reduces the number of parameters that must be estimated and improves numerical stability for sparsely populated classes. The pooled covariance is computed from the deviations of the profiling vectors from their corresponding class means,

$$\boldsymbol{\Sigma} = \frac{1}{N_{\mathcal{M}}} \sum_{c\in\mathcal{M}} \sum_{i:c_i=c} (\mathbf{t}_i - \boldsymbol{\mu}_c)(\mathbf{t}_i - \boldsymbol{\mu}_c)^T\,,$$

where $\mathcal{M}$ is the set of classes with enough profiling traces for a direct estimate (see below) and $N_{\mathcal{M}} = \sum_{c\in\mathcal{M}} N_c$ is the number of profiling traces in these classes.

Each leakage class is therefore represented by a multivariate Gaussian template,

$$\mathbf{t} \mid c \sim \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma})\,.$$

A class mean is estimated directly only when at least 30 profiling traces are available for that class. Classes with fewer observations are treated as insufficiently populated. Their mean vectors are obtained by extrapolation: for every POI, a straight line is fitted through the means of the sufficiently populated classes as a function of the Hamming weight (least squares, with each class weighted by its number of traces N_c), and the line is evaluated at the missing classes. Such a class is therefore not treated as impossible; a hypothesis that requires it simply receives a low likelihood when its predicted mean is far from the measured leakage.

Three independent template sets are constructed according to the three leakage resolutions, with 18 coarse, 24 fine, and 33 exact classes. The number of classes estimated directly and by extrapolation in the recorded profiling dataset is reported in Section V-A.

The resulting template sets are subsequently used in the corresponding stages of the hierarchical search. The coarse templates provide robust discrimination over the complete weight interval, the fine templates refine the most likely regions, and the exact templates are used for the final `float32`-level recovery.

D. Candidate Likelihood

During the attack phase, each candidate weight is evaluated according to how well its predicted leakage classes explain the measured power traces. For a candidate weight $\hat{w}$ and known input x_i, the hypothetical intermediate value is computed as

$$\hat{v}_i = \mathrm{float32}(\hat{w}x_i).$$

The corresponding leakage class is then obtained using the leakage model of the current search stage,

$$c_i(\hat{w}) = \mathrm{HW}\left(B_m(\hat{v}_i)\right),$$

where $m = 17$, 23, or 32 for the coarse, fine, or exact model, respectively.

Given the predicted class $c_i(\hat{w})$, the five-dimensional attack leakage vector $\mathbf{t}_i$ is evaluated using the corresponding Gaussian template. The log-likelihood contribution of trace i is

$$\ell_i(\hat{w}) = \log p\left(\mathbf{t}_i \mid c_i(\hat{w})\right).$$

Using the multivariate Gaussian model, this can be expressed as

$$\ell_i(\hat{w}) = -\frac{1}{2}\left(\mathbf{t}_i - \boldsymbol{\mu}_{c_i(\hat{w})}\right)^T \boldsymbol{\Sigma}^{-1}\left(\mathbf{t}_i - \boldsymbol{\mu}_{c_i(\hat{w})}\right) + K,$$

where K contains terms that are constant with respect to the candidate weight. Since the covariance matrix is shared across leakage classes, these constant terms do not affect candidate ranking and can be omitted during implementation. With the constant omitted, every contribution $\ell_i(\hat{w})$ is non-positive.

Assuming that the attack traces are independent given their leakage classes, the evidence from multiple traces is accumulated by summing their log-likelihood contributions,

$$L(\hat{w}) = \sum_{i=1}^{N_{\mathrm{attack}}} \ell_i(\hat{w}).$$

A correct candidate is expected to predict leakage classes that are consistently compatible with the measured traces. Incorrect candidates produce different hypothetical multiplication results and therefore different leakage-class sequences, resulting in lower accumulated likelihoods.

The candidate with the highest accumulated score is selected as

$$\hat{w}^\star = \arg\max_{\hat{w}} L(\hat{w}).$$

The same likelihood formulation is used in all three stages of the attack. The stages differ only in the candidate search space and in the resolution of the corresponding leakage templates.

E. Hierarchical Weight Recovery

To avoid an exhaustive search over the complete IEEE-754 single-precision space, the proposed attack uses a hierarchical recovery procedure composed of coarse, fine, and exact stages. Each stage evaluates candidate weights using the likelihood function introduced in Section IV-D, while progressively increasing the resolution of both the candidate search and the leakage model.

a) Coarse search.: The first stage searches the complete assumed weight interval

$$\hat{w} \in [-2, 2]$$

with a step size of 10^{-4}. This results in $40{,}001$ candidate weights. Each candidate is evaluated using the coarse template set based on the sign bit, exponent bits, and the 8 most significant fraction bits of the multiplication result. After likelihood accumulation over the available attack traces, the six highest-scoring separated peaks are retained. A minimum separation of 0.005 is enforced between selected peaks to avoid retaining several neighboring candidates corresponding to the same local maximum.

b) Fine search.: Each of the six coarse finalists is refined independently within a local interval of $\pm 10^{-4}$ around the corresponding coarse estimate. The local search uses a step size of 10^{-6}, resulting in 201 candidates per coarse finalist. The candidate likelihoods are recomputed using the fine template set, which includes 14 fraction bits in addition to the sign and exponent bits. The best candidate from each local region is retained, producing six refined candidates. The refined candidate with the highest fine-stage likelihood is then selected for exact recovery.

c) Exact search.: The best fine candidate is interpreted directly as an IEEE-754 single-precision value. Its neighboring representable `float32` values are enumerated within ± 512 units in the last place (ULPs). These values are additionally evaluated with both signs and scaled by 2^j for $j \in \{-10, \ldots, 10\}$, and candidates outside $[-2, 2]$ are discarded. The scaled variants $\pm \hat{w}\, 2^j$ have exactly the same fraction bits as $\hat{w}$, so their products differ from the true product only in the sign and exponent bits. The coarse and fine models cannot separate them reliably (Section V-B), and the exact stage must be able to reach the true weight even if the fine stage selected one of these look-alikes. For a fine result close to 1.43, this gives $1{,}025$ neighboring values, 11 scales that stay within $[-2, 2]$, and 2 signs, i.e., $22{,}550$ exact candidates.

Each remaining candidate is evaluated using the exact template set based on the complete 32-bit representation of the hypothetical multiplication result,

$$c_i(\hat{w}) = \mathrm{HW}\left(B_{32}\left(\mathrm{float32}(\hat{w} x_i)\right)\right).$$

The candidate with the highest accumulated exact-template likelihood is returned as the recovered weight,

$$\hat{w}^\star = \arg \max_{\hat{w} \in \mathcal{W}_{\mathrm{exact}}} L_{\mathrm{exact}}(\hat{w}).$$

The hierarchical procedure therefore reduces the search space progressively. The coarse stage identifies promising regions over the full weight interval, the fine stage resolves nearby numerical candidates, and the exact stage distinguishes neighboring IEEE-754 representations and power-of-two look-alikes to obtain a bit-precise `float32` estimate.

V. Experimental Results

A. Profiling Leakage

The profiling traces exhibit clear data-dependent leakage associated with the targeted floating-point multiplication. Using the full 32-bit Hamming-weight model defined in Section IV-A, the correlation analysis described in Section IV-B identifies several distinct leakage peaks within the analysis window.

The five selected points of interest are located at sample indices

$$\mathcal{P} = \{518,\ 569,\ 622,\ 657,\ 737\}.$$

The correlation is negative at all five POIs, and its absolute value lies between 0.74 and 0.93. This indicates a strong statistical dependence between the measured power consumption and the Hamming weight of the floating-point multiplication result.

Figure 1 shows the correlation for every sample of the analysis window together with the selected POIs. The first ≈ 400 samples show almost no correlation, which is consistent with the operations executed between the trigger and the start of the multiplication. The several strong and temporally separated peaks after that suggest that the intermediate value influences the measured leakage at multiple points during the software multiplication routine and the operations that follow it, such as storing the product and adding it to the neuron's sum.

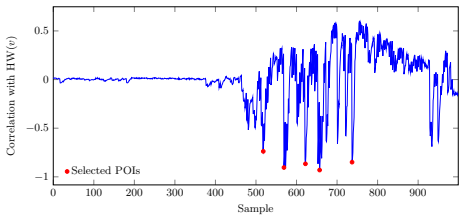


Fig. 1. Pearson correlation between the profiling traces and the Hamming weight of the full 32-bit product $v = \mathrm{float32}(wx)$ for every sample of the analysis window. The red markers are the five selected POIs.

The profiling data are subsequently divided into Hamming-weight classes for construction of the coarse, fine, and exact templates. Not all theoretically possible classes are sufficiently represented in the profiling dataset. By using a minimum threshold of 30 traces per class, 11 of the 18 coarse classes, 13 of the 24 fine classes, and 16 of the 33 exact classes are estimated directly from measured traces. The remaining class means are obtained through the extrapolation procedure described in Section IV-C.

Figure 2 shows the resulting class means at the first POI. For all three template sets, the means of the measured classes decrease almost linearly with the Hamming weight, which

supports the linear extrapolation of the missing classes. The error bars show the standard deviation of the noise at this POI. It is two to three times larger than the difference between neighboring class means, so a single trace carries little information and evidence from many traces has to be combined.

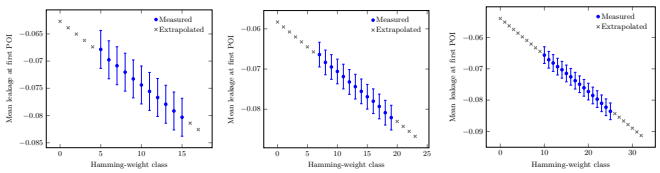


Fig. 2. Template means at the first POI for the coarse (left), fine (middle), and exact (right) template sets. Blue: classes estimated from at least 30 profiling traces, with error bars of one standard deviation of the noise. Gray: extrapolated classes.

These results confirm that the targeted multiplication produces sufficiently strong leakage for template construction, while also illustrating the increasing sparsity of Hamming-weight classes as more IEEE-754 bits are included in the leakage model.

B. Weight Recovery

We evaluate the proposed attack on the fixed target network described in Section III-B. The configured value of the attacked weight is 1.43, which is represented during execution by its nearest IEEE-754 single-precision value,

$$w^\star \approx 1.4299999.$$

The hierarchical recovery procedure described in Section IV-E is applied to the first 1,000 attack traces. The six coarse finalists are 1.4299, 0.7150, 0.3575, 0.0894, -1.4300, and 0.1787. Apart from the true weight, these are its look-alikes $w^\star/2$, $w^\star/4$, $w^\star/16$, $-w^\star$, and $w^\star/8$, whose products share the fraction bits of the true product. The fine stage ranks the finalist near the true weight highest and refines it to 1.4299961, which is 33 ULPs below the true value. The exact stage then recovers

$$\hat{w} = 1.4299999,$$

which matches the true single-precision representation of the target weight bit for bit (`0x3FB70A3D`). The same exact value is recovered when all 5,000 available attack traces are used.

The recovery error is therefore

$$|\hat{w} - w^\star| = 0,$$

when evaluated in the `float32` representation used by the implementation.

These results demonstrate that the proposed profiled template attack is able to identify the exact single-precision representation of the targeted first-layer weight from the measured power traces. The following sections further analyze how candidate likelihoods evolve as traces are accumulated and how the probability of successful recovery depends on the number of attack traces.

C. Candidate Convergence

To illustrate how evidence accumulates across attack traces, we analyze the cumulative log-likelihood of the candidates of every stage. For a candidate weight $\hat{w}$ and the first n attack traces, the cumulative score is defined as

$$L_n(\hat{w}) = \sum_{i=1}^{n} \ell_i(\hat{w}),$$

where $\ell_i(\hat{w})$ is computed with the templates of the respective stage.

Figure 3 shows L_n for the first 1,000 attack traces in acquisition order. For every stage, the true weight is compared with all wrong candidates of that stage: the complete coarse grid, the fine grids around all six coarse finalists, and all exact candidates. Instead of drawing every wrong candidate, the figure shows the best, the mean, and the worst score among them for every n. In the coarse and fine stages, candidates within 0.005 of the true weight are counted as correct, since these stages only have to reach the neighborhood of the true weight. In the exact stage, every candidate except the bit-exact true weight is counted as wrong.

Since every contribution $\ell_i(\hat{w})$ is non-positive (Section IV-D), all curves decrease as more traces are added. Candidate ranking is determined by the relative score, with the most plausible candidate corresponding to the highest, i.e., least negative, cumulative log-likelihood. The mean and the worst wrong candidate fall behind the true weight quickly: after 10 traces, about 97% or more of the wrong candidates of every stage already score below the true weight. The best wrong candidate stays close to the true weight much longer: the true weight scores higher than every wrong candidate after 427 traces in the coarse stage, 497 traces in the fine stage, and 446 traces in the exact stage. In the exact stage, all 1,024 neighboring `float32` values within ± 512 ULPs are already outscored after 7 traces. The strongest competitors are the power-of-two look-alikes ($w^\star/2$, $w^\star/4$, $w^\star/16$, and $w^\star/256$), whose products differ from the true product only in the exponent bits.

The coarse and fine stages do not need the true weight to score highest overall. The coarse stage only needs it among the six finalists, and the fine stage only compares the six refined finalists. The trace order shown here is also harder than average: the complete attack on these traces selects the look-alike $w^\star/256$ until 446 traces, whereas random subsets of 73 traces already succeed in 88% of the cases (Section V-D).

This convergence behavior also motivates the success-rate analysis in Section V-D, where the attack is repeated for different numbers of traces to quantify the probability of recovering the correct weight.

D. Success Rate

To quantify the number of attack traces required for reliable recovery, we evaluate the empirical success rate of the complete hierarchical attack for increasing trace counts. The trace counts N are 26 values spaced logarithmically between

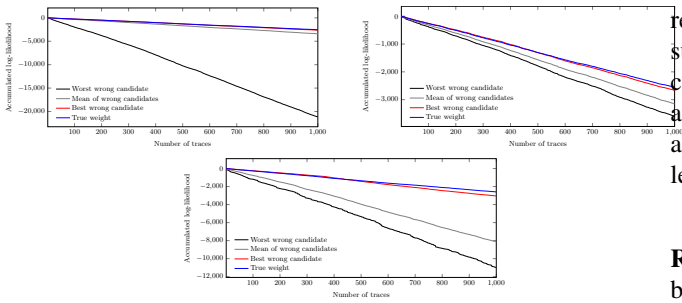


Fig. 3. Cumulative log-likelihood of the true weight and of the best, mean, and worst wrong candidate over the first 1,000 attack traces, for the coarse (top left, 39,900 wrong candidates), fine (top right, 1,005), and exact (bottom, 22,549) stage.

1 and 500. The experiment is repeated 100 times. In every repetition, 500 of the 5,000 attack traces are drawn in a random order, and the attack is run on the first N of them for every trace count, so the subsets are nested. A recovery is considered successful only when the recovered value equals the true `float32` weight bit for bit, i.e., $\hat{w} = w^{\star}$, the same criterion as in Section V-B.

The empirical success rate is defined as

$$\mathrm{SR}(N) = \frac{\#\{\text{successful recoveries using } N \text{ traces}\}}{100}.$$

Figure 4 shows the resulting success rate as a function of the number of attack traces. The attack exhibits a rapid improvement as additional traces are included, reaching approximately 64% success with 38 traces, 88% with 73 traces, and 95% with 138 traces. The success rate reaches 99% with 171 traces and 100% from 263 traces up to the largest evaluated count of 500.

These results indicate that the proposed profiling attack does not require the full set of 5,000 attack traces to recover the target weight reliably. Instead, the likelihood information accumulated from a few hundred measurements is sufficient to obtain consistent recovery in the evaluated setting.

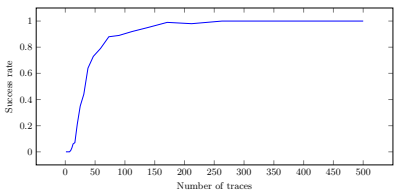


Fig. 4. Empirical success rate of the proposed weight-recovery attack as a function of the number of attack traces.

VI. Discussion

Interpretation of the Results The results indicate that the profiling model captures leakage associated with the targeted floating-point multiplication rather than with a single fixed network configuration. Although the non-target weights vary during profiling and are not included in the leakage model, the resulting templates remain sufficiently discriminative to recover the target weight in the fixed attack network. The success-rate analysis further shows that evidence accumulates consistently in favor of the correct candidate as more traces are used. This supports the assumption that the selected POIs and templates provide a meaningful statistical model of the leakage of

$$\mathrm{float32}(wx).$$

Role of the Hierarchical Search The hierarchical search balances robustness and precision during weight recovery. The coarse stage uses a reduced IEEE-754 leakage model to identify promising regions over the full weight interval, while the fine stage increases the leakage resolution to distinguish nearby candidates. The final stage then evaluates neighboring `float32` values using the complete 32-bit leakage model. This progression avoids applying the most detailed leakage model to the entire candidate space. It therefore reduces the computational cost of the search while preserving the ability to recover the exact floating-point representation of the target weight.

Limitations and Future Work The present study focuses on the first weight of the first layer on a single target platform, providing an initial evaluation of the proposed profiling methodology. A natural next step is to extend the analysis to multiple network configurations and deeper layers. Future work should also investigate the portability of the templates across different devices and measurement sessions, as well as the effect of larger or more balanced profiling datasets on sparsely populated Hamming-weight classes.

VII. Conclusion

This work presented a profiled template attack for recovering a single-precision neural-network weight from power side-channel measurements. The attack models the leakage of the floating-point multiplication between a known input and the unknown target weight, and uses templates constructed from randomized profiling configurations. A hierarchical coarse-to-fine-to-exact search enables efficient exploration of the candidate space and exact recovery of the target `float32` value. The experimental evaluation demonstrates successful weight recovery and shows that reliable recovery can be achieved with only a few hundred attack traces in the considered setting.

References

[1] H. Chabanne, J.-L. Danger, L. Guiga, and U. Kühne, "Parasite: Mitigating physical side-channel attacks against neural networks," in *International Conference on Security, Privacy, and Applied Cryptography Engineering (SPACE)*. Springer, 2021, pp. 148–167.

[2] M. Hashemi, S. Roy, D. Forte, and F. Ganji, "HWGN^2: Side-channel protected NNs through secure and private function evaluation," in *International Conference on Security, Privacy, and Applied Cryptography Engineering*. Springer, 2022, pp. 225–248.

[3] L. Batina, S. Bhasin, D. Jap, and S. Picek, "CSI NN: Reverse engineering of neural network architectures through electromagnetic side channel," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 515–532.

[4] L. Wei, B. Luo, Y. Li, Y. Liu, and Q. Xu, "I know what you see: Power side-channel attack on convolutional neural network accelerators," in *Proceedings of the 34th Annual Computer Security Applications Conference*, 2018, pp. 393–406.

[5] R. Joud, P.-A. Moëllic, S. Pontié, and J.-B. Rigaud, "A practical introduction to side-channel extraction of deep neural network parameters," in *International Conference on Smart Card Research and Advanced Applications*. Springer, 2022, pp. 45–65.

[6] H. Nozaki and K. Kobara, "Power analysis of floating-point operations for leakage resistance evaluation of neural network model parameters," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E107-A, no. 3, pp. 331–343, 2024.

[7] L. Pušká̌c, M. Benovič, J. Breier, and X. Hou, "Make shuffling great again: A side-channel-resistant Fisher–Yates algorithm for protecting neural networks," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2025.

[8] J. Breier, D. Jap, X. Hou, S. Bhasin, and Y. Liu, "Sniff: reverse engineering of neural networks with fault attacks," *IEEE Transactions on Reliability*, vol. 71, no. 4, pp. 1527–1539, 2021.

[9] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[10] L. Batina, S. Bhasin, D. Jap, and S. Picek, "Poster: Recovering the input of neural networks via single shot side-channel attacks," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 2657–2659.

[11] X. Yan, X. Lou, G. Xu, H. Qiu, S. Guo, C. H. Chang, and T. Zhang, "Mercury: An automated remote side-channel attack to Nvidia deep learning accelerator," *arXiv preprint arXiv:2308.01193*, 2023.

[12] Z. Lehocký, J. Breier, D. Jap, S. Bhasin, and X. Hou, "Side-channel analysis of OpenVINO-based neural network models," in *International Conference on Availability, Reliability and Security*. Springer, 2025, pp. 307–324.

[13] S. Chari, J. R. Rao, and P. Rohatgi, "Template attacks," in *Cryptographic Hardware and Embedded Systems – CHES 2002*, ser. LNCS, vol. 2523. Springer, 2003, pp. 13–28.

[14] "IEEE standard for floating-point arithmetic," *IEEE Std 754-2019 (Revision of IEEE 754-2008)*, pp. 1–84, 2019.

Chapter 11

Evaluation

11.1 Summary of Results

11.2 Impact

11.3 Future Work

Resumé

References

[1] Péter Horváth et al. “SoK: Neural Network Extraction Through Physical Side Channels”. In: *33rd USENIX Security Symposium (USENIX Security 24)*. 2024.

[2] Lejla Batina et al. “CSI NN: Reverse Engineering of Neural Network Architectures Through Electromagnetic Side Channel”. In: *28th USENIX Security Symposium (USENIX Security 19)*. 2019, pp. 515–532.

[3] Raphaël Joud et al. “A Practical Introduction to Side-Channel Extraction of Deep Neural Network Parameters”. In: *International Conference on Smart Card Research and Advanced Applications (CARDIS)*. Springer, 2022, pp. 45–65.

[4] Hanae Nozaki and Kazukuni Kobara. “Power Analysis of Floating-Point Operations for Leakage Resistance Evaluation of Neural Network Model Parameters”. In: *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences* E107-A.3 (2024), pp. 331–343.

[5] Cheng Gongye, Yunsi Fei, and Thomas Wahl. “Reverse-Engineering Deep Neural Networks Using Floating-Point Timing Side-Channels”. In: *57th ACM/IEEE Design Automation Conference (DAC)*. 2020, pp. 1–6.

[6] Jakub Breier et al. “SNIFF: Reverse Engineering of Neural Networks with Fault Attacks”. In: *IEEE Transactions on Reliability* 71.4 (2021), pp. 1527–1539.

[7] Suresh Chari, Josyula R. Rao, and Pankaj Rohatgi. “Template Attacks”. In: *Cryptographic Hardware and Embedded Systems – CHES 2002*. Vol. 2523. LNCS. Springer, 2003, pp. 13–28.

[8] Yuchen Tu et al. “Template Attacks Based on Two-dimensional Convolutional Neural Networks”. In: *2021 International Conference on Intelligent Computing, Automation and Applications (ICAA)*. IEEE, 2021, pp. 287–293. DOI: 10.1109/ICAA53760.2021.00060.

[9] Lingxiao Wei et al. “I Know What You See: Power Side-Channel Attack on Convolutional Neural Network Accelerators”. In: *Proceedings of the 34th Annual Computer Security Applications Conference (ACSAC)*. 2018, pp. 393–406.

[10] *IEEE Standard for Floating-Point Arithmetic*. IEEE Std 754-2019. IEEE. 2019. DOI: 10.1109/IEEESTD.2019.8766229.

Appendix A

Technical Documentation

A.1 Firmware

A.2 Capture Script and Configuration

A.3 Analysis Notebooks

A.4 Data Formats

Appendix B

Work Plan

Do i even need to do this?

B.1 Winter Semester

Week	Task

B.2 Summer Semester

Week	Task

B.3 Evaluation of the Work Plan

Appendix C

Contents of the digital medium

Registration number of the thesis in the information system: FIIT-XXXX-XXXXX

TODO

Contents of the digital medium (ZIP archive):

Folder	Contents
/firmware	target firmware (network forward pass)
/capture	trace capture script and configuration
/data	
/raw	captured ChipWhisperer projects
/processed	clipped traces used by the analysis
/analysis	leakage analysis and attack notebooks
/results	exported figures and tables
/thesis	LaTeX source of this thesis
/praca-pdf	
/praca.pdf	main part of the thesis
/prilohy.pdf	text appendices

Name of the submitted archive: TODO.